

Intermediate (wrap real logic behind a route) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper FastAPI/Pydantic), **Advanced** (how a real API tells 4xx from 5xx and never leaks internals). Read Basic first even if you already know FastAPI — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A route can call any regular Python function inside it — nothing special has to happen for "real logic" to run behind an endpoint. The new part in this exercise is: the *request* coming in should be a typed Pydantic model (not a raw dict), and if your real logic raises an error, the route needs to catch it and reply with the right HTTP status code instead of crashing.

Things to use:

- `from pydantic import BaseModel` then `class ChatRequest(BaseModel): message: str`.
- A route function that takes `request: ChatRequest` as a parameter — FastAPI parses and checks the incoming JSON against it automatically.
- `try / except` around the call to your real chat logic.
- `from fastapi import HTTPException` then `raise HTTPException(status_code=500, detail="...")`.

Intermediate Version

Declaring `def chat(request: ChatRequest):` is the whole trick — FastAPI reads that type hint, and before your function body runs even once, it parses the incoming request body as JSON, validates it against `ChatRequest`, and automatically replies with a 422 if it doesn't match. You never write that checking code yourself; it's the same idea as Doc04/06's tool-argument Pydantic models, just applied to an HTTP body instead of an LLM tool call.

Doc02's retry/error material comes back here in a new form: a bad-input problem is the *client's* fault (4xx — don't blindly retry the exact same request), an unexpected internal failure is the *server's* fault (5xx). `HTTPException` is FastAPI's way of raising an HTTP-shaped error from inside a route — it stops the function immediately and sends that exact status code and detail message back as the JSON reply.

The exact pieces:

- `class ChatRequest(BaseModel): message: str` — the input shape.
- `class ChatResponse(BaseModel): reply: str` — the output shape, same pattern as `HealthResponse` in the previous exercise.

- `@app.post("/chat", response_model=ChatResponse) def chat(request: ChatRequest) -> ChatResponse:`
- `try: reply = run_chat(request.message) except Exception as exc: raise HTTPException(status_code=500, detail="...") .`
- `request.message` — accessing a field on a validated Pydantic model, guaranteed to be a `str` by the time you read it.

Advanced Version

Two things a naive `except Exception: raise HTTPException(500, detail=str(exc))` gets wrong: it treats every failure as a 500 (even ones that are really the client's fault, like an empty message your chat logic refuses to handle), and it puts the raw exception text — which can include internal details, file paths, sometimes even a stray API key from an error message — directly into the client-facing reply.

The real design question: **what actually deserves a 500, versus a 4xx, and what should the client ever see about why it failed?** A missing/malformed field is already handled for you (422, by Pydantic, before your code runs at all). What's left for your own `try/except` is telling apart "your logic rejected this input on purpose" (400) from "something broke unexpectedly" (500) — and for the 500 case specifically, logging the real detail server-side while sending the client only a safe, generic message.

Pieces:

- A custom exception in your own chat logic (e.g. `class ChatInputError(Exception): pass`) raised on purpose for bad-but-well-formed input (like an empty message) — caught separately from a bare `Exception` and mapped to 400, not 500.
- `import logging; logger = logging.getLogger(__name__)` then `logger.exception("chat route failed")` inside the `except Exception` branch — logs the full traceback server-side, while the client still only gets a generic message.
- Never put `str(exc)` directly into an `HTTPException(detail=...)` for the 500 case — that's exactly the leak this exercise is testing for.

Difference between Basic, Intermediate, and Advanced: Basic names the pieces for wrapping logic behind a typed route with a status code on failure. Intermediate explains exactly how FastAPI's automatic 422 works and why `HTTPException` is the right tool for turning a caught error into a specific reply. Advanced questions whether "everything that goes wrong is a 500" is even correct, splits deliberate input-rejection (400) from genuine internal failure (500), and closes the leak of putting raw exception text into a client-facing reply — the exact thing `db_failure_and_docker` checks for later in this document.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
def ChatRequest(BaseModel): message: str
def ChatResponse(BaseModel): reply: str

route POST /chat, takes a ChatRequest:
    try:
        reply_text = call the real chat function with request.message
        return ChatResponse(reply=reply_text)
    except anything:
        raise a 500 HTTPException
```

Here's almost the whole thing — the `run_chat` stand-in is deliberately fake, swap in your Doc04 chat function later:

```
# chat_route_practice.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

def run_chat(message: str) -> str:
    # stand-in for your Doc04 chat logic
    return "You said: " + message

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest) -> ChatResponse:
    try:
        reply_text = run_chat(request.message)
        return ChatResponse(reply=reply_text)
```

```
except Exception:
    raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected output (`curl -X POST localhost:8000/chat -H "Content-Type: application/json" -d '{"message": "hi"}'`):

```
{"reply": "You said: hi"}
```

Intermediate Version

```
def ChatRequest(BaseModel): message: str
def ChatResponse(BaseModel): reply: str

function run_chat(message) -> str:
    if message is empty: raise ChatInputError
    ... real logic ...
    return reply text

route POST /chat, response_model=ChatResponse, takes a ChatRequest:
    try:
        return ChatResponse(reply=run_chat(request.message))
    except ChatInputError as exc:
        raise HTTPException(400, detail=str(exc))
    except Exception:
        raise HTTPException(500, detail="Something went wrong.")
```

```
# chat_route_practice.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

class ChatInputError(Exception):
    pass

def run_chat(message: str) -> str:
    if not message.strip():
        raise ChatInputError("message cannot be empty")
```

```

    return "You said: " + message

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest) -> ChatResponse:
    try:
        reply_text = run_chat(request.message)
        return ChatResponse(reply=reply_text)
    except ChatInputError as exc:
        raise HTTPException(status_code=400, detail=str(exc))
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")

```

Expected output for `{"message": ""}`: 400 `{"detail": "message cannot be empty"}`.

Expected output for a missing `message` field entirely: 422, from Pydantic, before `chat()` even runs.

Advanced Version

```

define ChatRequest, ChatResponse, ChatInputError as above

logger = logging.getLogger(__name__)

function run_chat(message) -> str:
    if message is empty: raise ChatInputError
    if message contains a magic "boom" test string: raise a plain RuntimeError (s
    return reply text

route POST /chat:
    try:
        return ChatResponse(reply=run_chat(request.message))
    except ChatInputError as exc:
        raise HTTPException(400, detail=str(exc))
    except Exception:
        logger.exception("chat route failed") # full traceback, server-side only
        raise HTTPException(500, detail="Something went wrong.") # no exc detail

```

Here's most of it — wire up the logger call yourself:

```

# chat_route_practice.py
import logging
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

logger = logging.getLogger(__name__)
app = FastAPI()

```

```

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

class ChatInputError(Exception):
    pass

def run_chat(message: str) -> str:
    if not message.strip():
        raise ChatInputError("message cannot be empty")
    if message == "boom":
        raise RuntimeError("simulated internal failure")
    return "You said: " + message

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest) -> ChatResponse:
    try:
        reply_text = run_chat(request.message)
        return ChatResponse(reply=reply_text)
    except ChatInputError as exc:
        raise HTTPException(status_code=400, detail=str(exc))
    except Exception:
        # your turn: log the real exception here with logger.exception(...)
        # then raise HTTPException(500, detail="...") with a generic message only
        ...

```

Send `{"message": "boom"}` and confirm: the client gets a plain 500 message, and the *real* `RuntimeError: simulated internal failure` traceback only shows up in your terminal, never in the HTTP response.

Difference between Basic, Intermediate, and Advanced: Basic catches everything as one generic 500. Intermediate splits a deliberate input problem (`ChatInputError` → 400) from everything else (→ 500). Advanced adds the piece that actually matters for production: a genuine internal crash gets logged in full detail server-side, while the client only ever sees a safe, generic message — proven here with a fake `RuntimeError` you can trigger on command.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)